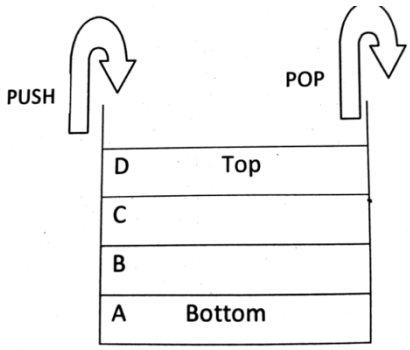
المكدس

Stack

يعتبر المكدس من أهم المفاهيم في علم الحاسب الآلي لما له من دور أساسي في برمجة الحاسوب. وهو هيكل بياني من النوع الخطي Linear structures يحتوي على نهاية واحدة يتم من خلالها اجراء عمليات الاضافة والحذف. وتسمى عملية اضافة عنصر الى المكدس بـ push، وتسمى عملية حذف عنصر من المكدس بـ pop، وللمكدس مؤشر يسمى top حيث ان هذا المؤشر يشير الى آخر عنصر دخل إلى المكدس، وبمعنى آخر يشير المؤشر top إلى أول عنصر سوف يخرج او يحذف من المكدس. وعندما يكون المكدس فارغاً فان قيمة المؤشر top = صفر.



وعند إضافة عنصر جديد الى المكدس يتم إضافة 1 إذا قيمة المؤشر top أي top=top+1 ليشير إلى موقع أعلى عنصر في المكدس. ويوضح الشكل التالي طريقة تريب البيانات داخل المكدس. ويتميز المكدس بخاصية أو مبدأ (Last in first out) (الداخل أخيراً يخرج أولاً) (LIFO) ويمكن تمثيل المكدس على شكل مصفوفة أحادية.

عندما يكون المكدس محدد الحجم فإنه يجب التحقق من حالته والتأكد من أنه غير ممتلىء عند إضافة عنصر جديد إليه. لأن محاولة الإضافة إلى مكدس ممتلىء سيؤدي ذلك الي ما يعرف بحالة over-flow. وكذلك عند حذف عنصر من المكدس يجب التأكد من حالته والتحقق من أنه غير فارغ، أي أن top لا تساوي صفر.

مثال: إذا تم إدخال البيانات 1 2 3 4 5 وإخراجها (من وإلى المكدس) حسب عمليات الإضافة والحذف التالية SSUSUSSUUU . حيث تشير S إلى عملية push أو إضافة، وتشير U إلى عملية pop أو حذف فإن المخرجات ستكون الشكل التالي: 1 4 5 3 2

العمليات على المكدس

- اختبار كون المكدس ممتلئاً (isfull)
- اختبار كون المكدس فارغاً (isempty)
- إضافة عنصر إلى مكدس (add)
- حذف عنصر من مكدس (delete)

١- يتم اختبار كون المكدس ممتلئاً بالدالة الإجرائية التالية :

```
boolean isfull(int top)
    {
  if (top == MAX_STACK_SIZE-1) return true;
    else return false;
                }
```

٢- يتم اختبار كون المكدس فارغاً بالدالة الإجرائية التالية :

```
boolean isempty(int top)
    {
  if (top== -1) return true;
   else return false;
      }
```

٣- إضافة عنصر إلى مكدس يتم بالدالة الإجرائية التالية :

```
void add(int *top, element item)
{
 if (isfull (*top) == true){
printf("error : stack is full");
return ;
   }
stack[++*top]=item;
            }
```

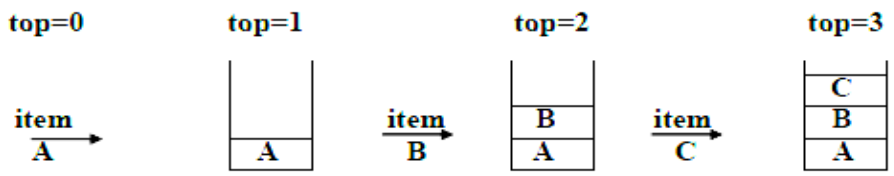
٤- حذف عنصر من مكدس يتم بالدالة الإجرائية التالية :

```
element delete(int *top)
    {
  if (isempty (*top) == true){
     printf("error : stack is empty");
     return ;
       }
     return stack[(*top)--];
         }
```

هناك عمليتين اساسيتين لادارة المكدس ((الداخل اولا يخرج آخرا)) **LIFO** :-

- عملية الدفع (**Push**) :

يزداد المؤشر بمقدار واحد على شرط كونه اقل من حجم المكدس.

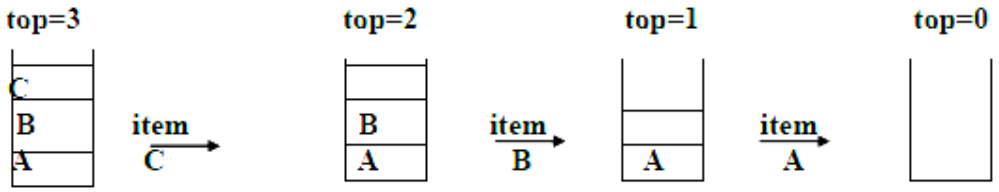


size= 3

stack is over flow المكدس ممتلئ

- عملية السحب (**Pop**) :

يتناقص المؤشر بمقدار واحد على شرط كون المكدس غير فارغ.



stack is under flow المكدس فارغ

استخدام المكدس في حل بعض المسائل

نتعرف في هذا الجزء عن كيفية استخدام المكدس في حل بعض المسائل الرياضية. نفترض أن لدينا التعبير الرياضي التالي:

7 - ((x * ((x + y) / (j – 3)) + y) / (4 – 2.5))

إذا أردنا التأكد من أن عدد الأقواس (تداخل الأقواس) صحيح، أي نريد التأكد من أن:

- عدد الأقواس اليمنى يساوي عدد الأقواس اليسرى.
- كل قوس أيمن (إقفال) يكون مسبوقاً بقوس أيسر (فتح) مقابل له.

فالتعابير التالية: A+B(أو (A+B)) لا يتحقق فيها الشرط الأول.

أما التعابير: A+B (-C) أو (A+B))-(C+D) فلا يتحقق فيها الشرط الثاني.

لحل هذه المشكلة والتأكد من صحة عدد الأقواس المستخدمة في أي تعبير، نقوم بإستخدام متغير يعبر عن عدد الأقواس عند أي نقطة من التعبير وليكن Parenthesis Counter (Pc) بحيث تكون قيمة Pc = عدد الأقواس اليسرى – عدد الأقواس اليمنى.

وفي حال أن عدد الأقواس في التعبير متساو فإن أي من الشرطين التاليين يكون محققاً:

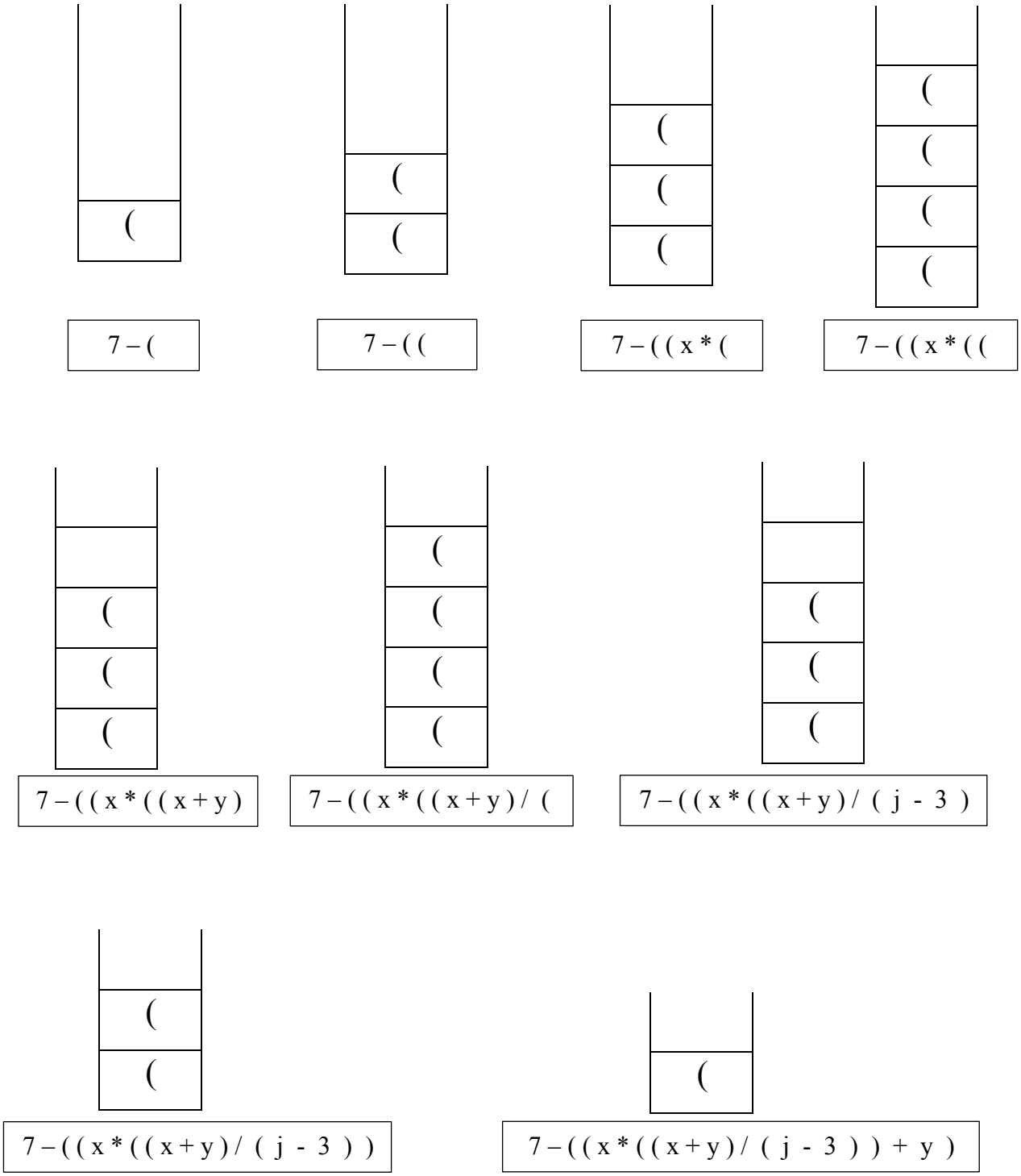
- عدد الأقواس في نهاية التعبير = صفر (قيمة Pc عند نهاية التعبير= صفر)،
- عدد الأقواس عند أي نقطة من التعبير يجب أن <u>لا يكون</u> سالباً. أي أن قيمة Pc يجب أن لاتكون سالبة. وهذا يعني أنه لايوجد قوس أيمن سابق للقوس الأيسر المقابل له.

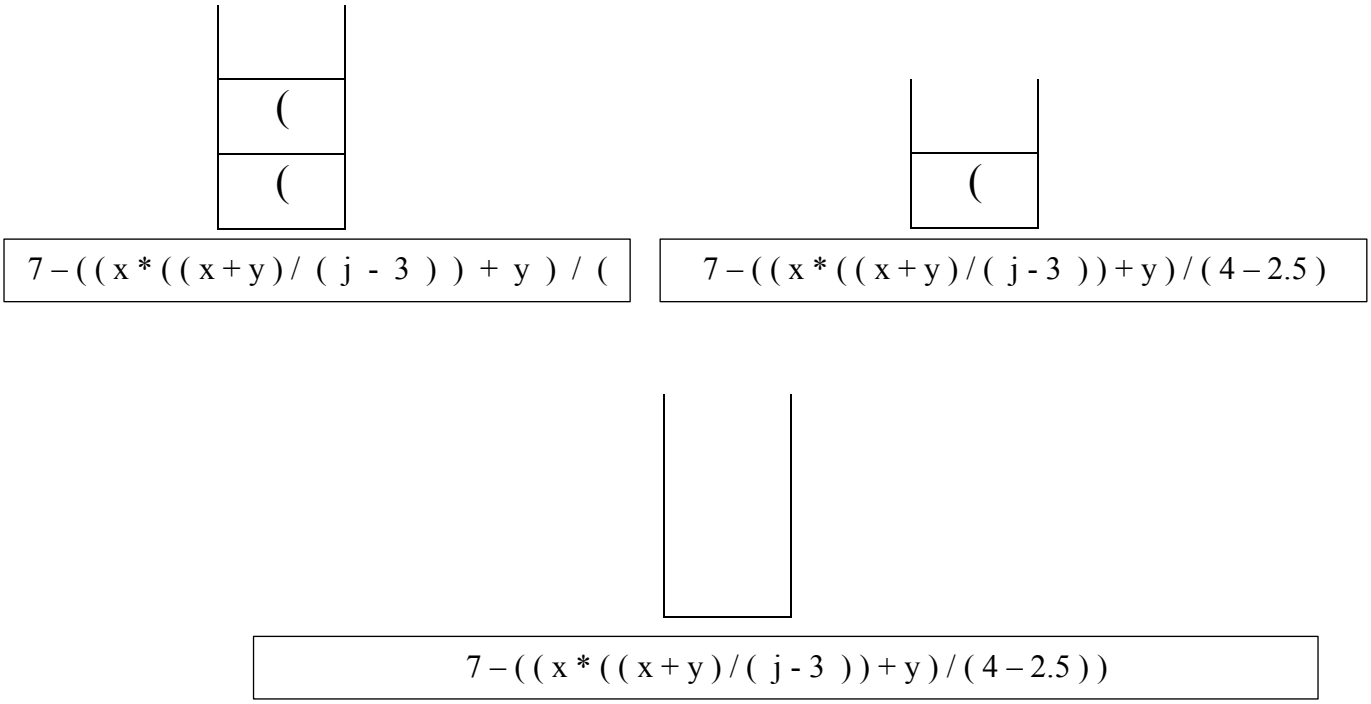
وباستخدام التعبير أعلاه، يمكن التأكد من صحة هذه الشروط.

```
7 - ( ( x * ( ( x + y ) / ( j – 3 ) ) + y ) / ( 4 – 2.5 ) )
0 0 1 2 2 2 3 4 4 4 4 3 3 4 4 4 4 3 2 2 2 1 1 2 2 2  2 1 0
```

وللتأكد من صحة عدد الأقواس، نقوم بالمرور على التعبير من اليسار إلى اليمين. عند مقابلة قوس أيسر (فتح) نجري عمية إضافة إلى المكدس Push وعند مقابلة قوس أيمن (إقفال) نجري عملية حذف Pop من المكدس. وعند الوصول إلى نهاية

التعبير يجب أن يكون المكدس فارغاً وإلا فإن التعبير غير صحيح. وفيما يلي توضيح لخطوات حل المثال أعلاه بإستخدام المكدس.





تمثيل المكدس برمجياً

قبل كتابة البرنامج الذي يمثل المكدس، يجب أولاً تحديد الطريقة التي سيتم بها تمثيل هذا الهيكل من البيانات. وبما أن المكدس هو عبارة عن مجموعة مرتبة من العناصر، نجد أنه يمكن استخدام المصفوفة لتمثيل المكدس برمجياً بالشكل الملائم. وعلى الرغم من أن خصائص المصفوفة تختلف عن خصائص المكدس، فالمصفوفة عبارة عن هيكل ثابت الحجم، في حين أن المكدس يعتبر هيكل متغير (ديناميكي)، حيث أن حجمه (عدد عناصره) يتغير وذلك من خلال عمليتي الإضافة والحذف، إلا أنه يمكن إستخدام المصفوفة لتمثيل المكدس وذلك بتعريف المكدس ليكون على شكل مصفوفة، ويكون حجم المصفوفة مساوياً للحجم الأقصى لعناصر المكدس المطلوب تمثيله. وتكون إحدى نهايتي المصفوفة هي قاعدة المكدس bottom، بحيث لايتم إجراء أي عملية إضافة أو حذف عند هذه القاعدة أو النهاية. في حين تستخدم النهاية الأخرى للمصفوفة لإضافة وحذف عناصر المكدس. ويكون المكدس فارغاً عندما لايحتوي على أي عناصر، عند هذه الحالة تكون قيمة top تساوي صفر. برنامج C++ التالي يبين كيفية تمثيل المكدس برمجياً وكيفية تنفيذ عمليتي الإضافة والحذف وكذلك اختبار ما إذا كان المكدس ممتلاءً أم فارغاً قبل تنفيذ عمليتي الإضافة والحذف.

فعند إجراء عملية إضافة push بيانات الى المكدس، يجب التأكد أولاً من أن هذا المكدس غير ممتلئ حتى لا يتم الوقوع في حالة overflow والتي تعني محاولة الإضافة الي مكدس ممتلئ

حيث لا يسمح بتجاوز عدد العناصر المحددة مسبقاً. وعند تنفيذ عملية حذف pop بيانات من المكدس، يجب التأكد أولاً من أن هذا المكدس غير فارغ حتى لا يتم الوقوع في حالة underflow والتي تعني محاولة الحذف من مكدس فارغ.

```
#include <iostream>
using namespace std;
#define max 3
int stack[max];
int top = 0;
void push (int a)
{ if ( top == 3)
        cout <<" stack is full "<< a <<" cannot be
added "<<endl;
    else
    {
        top ++;
        stack[top]=a;
    }
}
void pop()
{ int x;
    if (top == 0)
        cout <<" stack is empty"<<endl;
    else
    {   x=stack [top];
        top --;
        cout <<"deleted " << x<<endl;
    }
}
int main ()
{
    push (1);
    push (2);
    push (3);
    push (4);
    pop ();
    pop ();
    pop ();
    pop ();
 return 0;
}
```

أهم تطبيقات المكدس

1. معالجة التطبيقات التي تحتوي على برامج فرعية أو دوال functions، فعند استدعاء برنامج فرعي داخل البرنامج الرئيسي، فان ذلك يتطلب تخزين عنوان الايعاز التالي بعد إيعاز الاستدعاء لكي يستطيع البرنامج الرئيسي تنفيذ البرنامج الفرعي والعودة بشكل صحيح الى موقع الإيعاز التالي.
2. يستخدم المكدس كهيكل لتخزين البيانات التي تحتاج عملية معالجتها إلى الاسترجاع بصورة معكوسة (بترتيب معكوس).
3. استخدام المكدس في معالجة التعابير الحسابية.

التعابير الحسابية Arithmetic expressions

تكتب التعابير الحسابية بثلاث صيغ؛ التوسط، التأخر والتصدر Infix, postfix, and prefix

1- infix (operand operator operand) صيغة infix notation

حيث أن اشارة العملية الحسابية تتوسط العوامل، وهذه هي الصيغة الاعتيادية مثل:

x/20 ، A-B ،3+4

2- prefix (operator operand operand) صيغة prefix notation

حيث تسبق اشارة العملية الحسابية العوامل اي تكون الاشارة من جهة اليسار مثل:

/x20 , -AB , +34

3- postfix (operand operand operator) صيغة postfix notation

حيث تكون اشارة العملية الحسابية تلحق العوامل اي تكون الاشارة من جهة اليمين مثل:

x20/ , AB- , 34 +

Infix = A+B
Prefix= +AB
Postfix= AB+

والتسميات نابعة من موقع المؤثر بالنسبة للمعاملات. إن ترميزى التصدر والتأخر ليسا صعبين أو معقدين كما يبدوان للوهلة الاولى.

فالتعريف التجريدي للجمع كدالة، مثلاً add(A,B) يستخدم المؤثرadd كمتصدر للمعاملات A وB.

إن استخدام ترميز التوسط مثل A+B*C يتطلب معرفة اي من العمليتين + أو* يجب ان تنفذ أولاً، وللأننا نعرف أن الضرب يسبق الجمع فإننا نتوقع إجراء عملية الضرب أولاً مالم تستخدم الأقواس لتعني غير ذلك.

نفترض أننا نريد كتابة A+B*C بالترميز المتأخر postfix سنقوم بتطبيق قواعد الاسبقيات، سنحول الجزء، الذي سينفذ أولاً (أي الضرب)، ثم الذي يليه، الخطوات التالية توضح ذلك وسنستخدم الاقواس للتوضيح فقط.

تحويل الضرب A+(B*C)

تحويل الجمع A+(BC*)

الشكل المتأخر A(BC*)+

ABC*+

مثال: مراعاة وجود الأقواس

شكل متوسط	(A+B)*C
تحويل القوس	(AB+)*C
تحويل الضرب	(AB+)C*
الشكل المتأخر	AB+C *

إذا استخدم تعبير يحوي أكثر من عمليه لها نفس الأسبقية دون استخدام الاقواس فإن ترتيب إجراء العمليات يكون من اليسار إلى اليمين، أي أن A+B+C يعني (A+B)+C. و A$ B$ C يعني (A $ B) $c حيث $ هو مؤثر الأس.

أما استخدام الأقواس في التعبير فيؤدي إلى تجاوز مسألة الأسبقية. فيما يلي عدد من التعابير بشكلي الترميز المتوسط والمتأخر.

Ex: convert the following expression to prefix & postfix?

a + b*c /d + e 1= b*c
1

a +1/d +e 2=1/d
2

a +2 + e 3= a +2
3

3+e

Prefix
3+e
+3e
+a+2e
++a2e
++a1/de
++a/1de
++a/b*c de
++a/* b c d e

Postfix
3+e
3e+
a +2e+
a2+e+
a 1/d +e+
a 1d /+e+
a b*c d /+e+
abc*d /+e+

	متوسط INFIX	متأخر POSTFIX
1.	A+B	AB+
2.	A+B-C	AB+C-
3.	(A+B)*(C-D)	AB+CD -*
4.	A$ B*C-D+E/F/(G+H)	AB$C*D-EF/GH+/+
5.	((A+B)*C-(D-E))$(F+G)	AB+C*DE—FG+$
6.	A-B/(C*D$E)	ABCDE$*/-

إن القواعد التي تتبع في التحويل للشكل المتأخر postfix هي نفسها التي تتبع للتحويل للشكل المتقدم prefix سوى أننا نضع المؤثر قبل المعاملات لا بعدها.

مثال:

	متوسط INFIX	متقدم PREFIX
1.	A+B	+AB
2.	A+B-C	-+ABC
3.	(A+B)*(C-D)	*+AB-CD
4.	A$B*C-D+E/F/(G+H)	+ - * $ABCD//EF+GH
5.	((A+B)*C-(D-E))$(F+G)	$ -*+ABC – DE + FG
6.	A-B/(C*D$E)	- A/B*C $DE

ملاحظة: القواعد التي يجب اتباعها في تحويل التعبير إلى الترميز المتأخر تتمثل فقط في أن العلميات الأعلى أسبقية تحول أولاً، ثم تعامل لاحقاً على أنها معامل مفرد. ولتنفيذ تعبير حسابي مكتوب بصيغة infix فإن العمليات تنفذ من جهة اليسار إلى اليمين وحسب أعلى أسبقية للعملية الحسابية.

Ex: if postfix expression is ABC/D*+ show the phases of executing this expression and find finish value when; A=5, B=4, C=2 and D=2.

Solution:

Postfix	current operator	current operant
ABC/D*+	/	1=B/C
A1D*+	*	2= 1*D
A2 +	+	3=A+2
3		

3=A+2
=A+1*D
=A+B/C*D
=5+4/2*2
=9

ملاحظة: يجب ان يكون عدد المعاملات في كل تعبير حسابي أكثر من عدد العمليات الحسابية بمقدار واحد، فإذا أعطينا لكل معامل قيمة (1+) ولكل عملية حسابية قيمة (1-)، فيجب ان تكون الدرجة النهائية rank لأي تعبير حسابي تساوي 1

Ex: find the rank of the following expression and show if it is a valid expression or not?

Infix	postfix	rank	case
1- a+*b	ab*+	0	invalid
2- a+b*c	ab+c*	1	valid
3- a+b/d-e	abd/+e-	1	valid
4- ab+c	abc+	2	invalid

ملاحظة إن استخدام الأشكال المتقدمة أو المتأخرة لا يتطلب استخدام الأقواس، فمثلاً:

متوسط	متأخر
A+(B*C)	ABC*+
(A+B)*C	AB+C*

إن ترتيب المؤثرات في الاشكال المتأخرة هو الذي يعبر حقيقة عن ترتيب إجراء العمليات مما يجعل استخدام الأقواس غير ضروري ويعتبر المكدس هو الهيكلية المناسبة لإيجاد قيم التعابير المكتوبة بالأشكال المتقدمة والمتأخرة.

Convert from infix to post & prefix.

(A + B) * (C - D) $ E*F

(A + B) * (c + $(D - E) + F) - G

A+ (((B - C)*(D - E) + F) / G) $(H-J)

Convent from postfix to infix:

AB + C -

ABC + -

AB - C+DEF - +$ ----------->(A-B+C) $ (D+(E-F))

ABCDE - + $ *EF * - --------> A*(B$ (C+ (D-E))) - (E *F)

خوارزمية تحويل التعبير من الشكل المتأخر postfix إلى الشكل المتوسط infix

1- إجعل المكدس فارغاً.

2- إقرا التعبير من اليسار إلى اليمين، طالما أن التعبير لم ينتهي قم بالآتي:

أ- في حال قراءة معامل ادفع به إلى المكدس.

ب-في حال قراءة مؤثر فإن معاملاه يكونان هما قمة المكدس، فقم بالآتي:

i. إنزع المعاملان

ii. قم بإنشاء سلسلة يكون فيها المؤثر بين المعاملان.

iii. إدفع بالسلسلة الناتجة إلى المكدس.

3- إرجع إلى الخطوة 2 إلى أن ينتهى التعبير.

4- عند الإنتهاء من قراءة التعبير تكون السلسلة الناتجة هي المطلوب.

مثال: حول التعبير التالي إلي الشكل المتوسط abc - + de - +

التعبير	المكدس	العملية
a	a	يتم تتبع خطوات الخوارزمية أعلاه
b	a,b	
c	a,b,c	
-	a,(b-c)	
+	(a+(b-c))	
d	(a+(b-c)),d	
e	(a+(b-c)),d,e	
-	(a+(b-c)),(d-e)	
+	(a+(b-c))+(d-e)	

الناتج : (a+(b-c))+(d-e)

إستخدام المكدس لإيجاد قيمة تعبير بالشكل المتأخر

كل مؤثر في التعبير بالشكل المتأخر يستخدم المعاملان السابقان له ومن الممكن أن يكون أحد هذين العاملين ناتج إجراء عملية سابقة ويمكن استخدام المكدس لإيجاد قيمة التعبير باتباع الخطوات التالية:

1- إجعل المكدس فارغاً.

2- إقرا التعبير من اليسار إلى اليمين مع مراعاة الآتي:

أ- في حال قراءة معامل ادفع به إلى المكدس.

ب-في حال قراءة مؤثر فإن معاملاه يكونان هما قمة المكدس، فقم بالآتي:

i. إنزع المعاملان

ii. نفذ العملية على المعاملان

iii. إدفع بالنتيجة إلى المكدس

3- إرجع إلى الخطوة 2 إلى أن ينتهى التعبير.

مثال:

أوجد قيمة التعبير المتأخر التالي:

623+-382/+*2$3+

الحل:

سنبين محتويات المكدس بعد قراءة كل حد من حدود التعبير كما سنبين معاملي القمة بعد كل عملية وقيمة التعبير وقتها .سنقوم بدفع كل معامل في التعبير عند قراءته إلى المكدس.

Symbol الحد	Opnd1 المعامل الأول	Opnd2 المعامل الثاني	Value القيمة	Stack المكدس
6				6
2				6 , 2
3	2	3		6 , 2 , 3
+	2	3	5	6 , 5
-	6	5	1	1
3	1	3	1	1 , 3
8	3	8	1	1 , 3 , 8
2	8	2	1	1 , 3 , 8 , 2
/	8	2	4	1 , 3 , 4
+	3	4	7	1 , 7
*	1	7	7	7
2	7	2	7	7 , 2
$	7	2	49	49
3	49	3	49	49 , 3
+	49	3	52	52

إذا قيمة التعبير هي 52

صيغة التوسط للتعبير أعلاه كما يلي:

infix ((6-(2+3))*(3+(8/2)))$2+3

تحويل التعبير من الترميز المتوسط إلى المتأخر

الخوارزمية التالية تبين كيفية تحويل تعبير رياضي مكتوب بالشكل المتوسط إلى الشكل المتأخر.

1. إجعل المكدس خالياً.
2. إقراء التعبير من اليسار إلى اليمين وستواجه الإحتمالات التالية:

أ- إذا كان المقرؤ معامل، ضعه في السلسلة التي تمثل التعبير المتأخر مباشرة.

ب- إذا كان المقروء مؤثر، قم بالآتي:

i. إذا كان المكدس خالياً، إدفع به إلى المكدس

ii. إذا كان المكدس يحوي قوس أيسر في قمته، إدفع بالمؤثر إلى المكدس

iii. إذا كان المكدس يحوي في قمته مؤثر فقارن بين المؤثرين وقم بالآتي:

a. إذا كان المؤثر الموجود في القمة **أقل أسبقية** من المؤثر الجديد، إدفع بالمؤثر الجديد إلى القمة.

b. إذا كان المؤثر الموجود في القمة **ذو أسبقية** على المؤثر الجديد فإن مؤثر القمة ينفذ أولاً، أي يكتب في السلسلة التي تمثل التعبير المتأخر، ثم يقارن المؤثر الجديد مع القمة الجديدة وهكذا.....

c. اذا كان المؤثران **لهما نفس الأسبقية**، ننفذ المؤثر الموجود في القمة أولاً ثم ندفع بالمؤثر الجديد إلى المكدس.

(ج) إذا كان المقرؤ قوس أيسر فادفع به إلى المكدس

(د) إذا كان المقرؤ قوس أيمن فقم بالآتي:

i. إنزع كل المؤثرات الموجودة في المكدس وضعها في التعبير المتأخر إلى أن تصل إلى قوس أيسر.

ii. إلغ القوس الأيسر والأيمن.

3. إذا اكتملت قراءة كل الحدود في التعبير ولازال المكدس يحتوي على بعض المؤثرات، انزعها وأضفها إلى التعبير.

مثال:

حول التعبير A*B+C إلى الشكل المتأخر.

	Symbol	Postfix String	Stack
1	A	A	
2	*	A	*
3	B	AB	*
4	+	AB*	+
5	C	AB*C	+
6		AB*C+	

الشرح:

السطور 5,3,1 تبين قراءة عوامل تضاف مباشرة إلى التعبير المتأخر في السطر 2 تمت قراءة مؤثر (*) ولأن المكدس خالي فقد تم الدفع به إلى المكدس، في السطر 4 تمت قراءة مؤثر الجمع (+) لذا فقد قورن بالمؤثر الموجود في قمة المكدس وهو (*) ولأن قمة المكدس في هذه الحالة ذات اسبقية على المؤثر الجديد (+) فقد تم نزع مؤثر الضرب (*) واضافته للتعبير المتأخر، فأصبح المكدس خالياً فتم دفع مؤثر الجمع فيه. في السطر 6 انتهى التعبير ومازال المكدس يحتوي بعض المؤثرات فتم نزعها واضافتها إلى التعبير المتأخر.

مثال: حول التعبير (A+B)*C إلى الشكل المتأخر.

	Symbol	Postfix String	Stack
1	(		(
2	A	A	(
3	+	A	(+
4	B	AB	(+
5	)	AB+	
6	*	AB+	*
7	C	AB+C*	*
8			

في السطر 5 تمت قراءة قوس أيمن فتم نزع المؤثر + والقوس الأيسر) ، ثم اضيف المؤثر للتعبير المتأخر وثم إلغاء القوسين.

مثال: حول التعبير((A-(B+c))*D)$(E+F) إلى الشكل المتأخر.

	رمز	التعبير المتأخر	المكدس
1	(		(
2	(		((
3	A	A	((
4	-	A	((-
5	(	A	((-(
6	B	AB	((-(
7	+	AB	((-(+
8	C	ABC	((-(+
9	)	ABC+	((-
10	)	ABC+-	(
11	*	ABC+-	(*
12	D	ABC+-D	(*
13	)	ABC+-D*	
14	$	ABC+-D*	$
15	(	ABC+-D*	$(
16	E	ABC+-D*E	$(
17	+	ABC+-D*E	$(+
18	F	ABC+-D*EF	$(+
19	)	ABC+-D*EF+	$
الناتج		ABC+-D*EF+$	

Ex: convert the following expression from infix to postfix using stack. A+B/ C* D – E

Infix	**stack**	**postfix**	**rank**
A		A	0
+	+	A	1
B	+	AB	1
/	+/	AB	2
C	+/	ABC	2
*	+*	ABC/	2
D	+*	ABC/D	2
-	-	ABC/D*+	1
E	-	ABC/D*+E	1
		ABC/D*+E-	1 valid

Ex: convert the expression ((A+B/ C –D)/E from infix notation to postfix notation. using stack?

Infix	**stack**	**postfix**
(	(	
(	((	
A	((	A
+	((+	A
B	((+	AB
)	(	AB+
/	(/	AB+
C	(/	AB+C
-	(-	AB+C/
D	(-	AB+C/D
)		AB+C/D-
/	/	AB+C/D-
E	/	AB+C/D-E/
		AB+C/D-E/